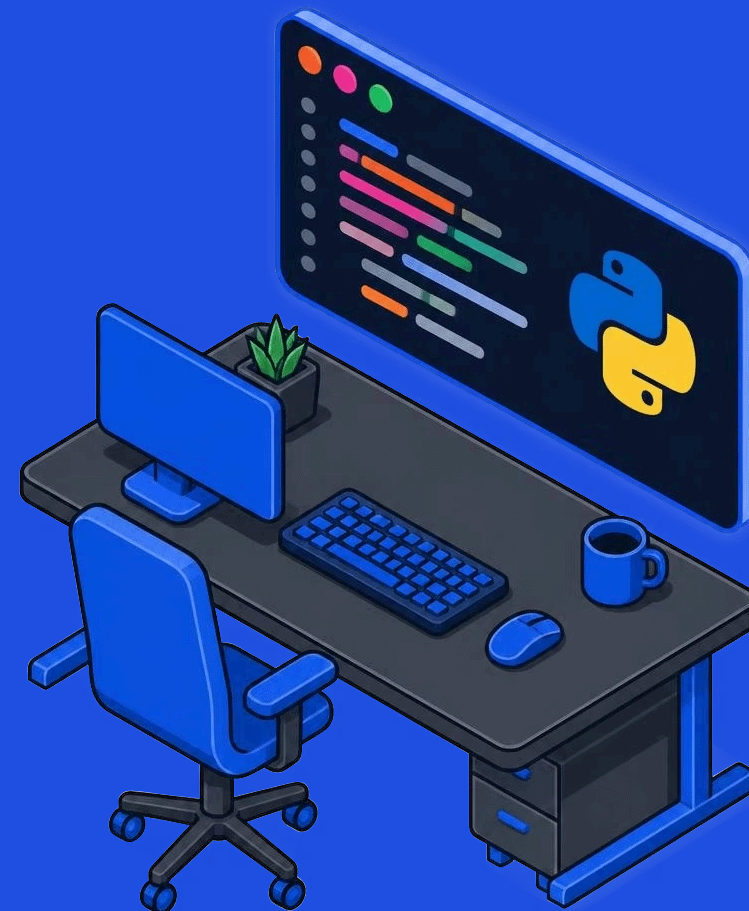


Python 完整教學

從安裝到進階實戰

建立紮實觀念，寫出優雅且具維護性的 Python 程式

系統化學習指南 · 零基礎快速上手 · 涵蓋語法、工具到專案架構



學習地圖：四個階段，一步步精通

循序漸進建立核心技能，每一步都有明確目標

15 個單元

理論說明搭配實戰範例與最佳實踐

階段 01

環境建置與工具準備

安裝 Python 3.12+、配置 VS Code，建立專屬虛擬環境。

階段 02

初階基礎語法與結構

變數、型別、條件判斷、迴圈與四種核心資料結構。

階段 03

自訂函式、檔案讀寫、例外處理與第三方套件生態。

階段 04

進階架構與專案實務

物件導向、裝飾器、非同步運算、實戰、單元測試與打包發佈。



STEP 01 官方安裝指引

第一步：下載並安裝 Python

建議使用 3.12 以上的最新穩定版本，獲得最佳效能與最新語法支援。

1

前往官方網站下載

至 python.org 下載對應作業系統的最新安裝檔。

2

Windows 務必勾選 PATH

安裝畫面最下方務必勾選「Add python.exe to PATH」，避免指令找不到。

3

macOS 與 Linux 快速安裝

macOS 可用官方安裝檔或 Homebrew (`brew install python`)；Linux 多數已內建，也可用 `apt install python3` 安裝。

4

驗證安裝狀態

開啟終端機執行 `python --version`，顯示版本號即代表安裝成功。

設定編輯器與虛擬環境

編輯器選擇：推薦 VS Code

安裝微軟官方「Python」擴充套件，可獲得語法高亮、智慧補齊與中斷點除錯。初學者也可選 Thonny 或 PyCharm 社群版。

```
hello.py
print("Hello, Python!")
執行：python hello.py
```

隔離專案：虛擬環境實務

不同專案依賴的套件版本常有衝突，使用 `venv` 為每個專案建立獨立空間，避免全域污染。

```
python -m venv .venv
source .venv/bin/activate
pip install requests
pip freeze > requirements.txt
```

變數與基本資料型別

Python 是動態型別語言，變數不需宣告型別。

四大基本純量型別

`str` 文字字串、`int` 整數、`float` 浮點數與 `bool` 布林值 (True/False)。用 `type()` 查看型別。

算術運算子：`+`、`-`、`*`、`/`、`//`、`%`、`**`。

字串格式化與輸入轉換

```
name, age, pi, ok = "Amy", 20, 3.14, True
print(f"{name} 今年 {age} 歲")
raw_input = input("請輸入購買數量: ")
count = int(raw_input)
total = count * 150
print(f"總金額為: {total}")
```

流程控制：條件與迴圈

Python 以 4 個空格縮排表示區塊；for i in range(5) 固定走訪 5 次，while 在條件成立時重複。

條件分支：if / elif / else

```
score = 85
if score >= 90: grade = "A"
elif score >= 80: grade = "B"
else: grade = "C"
```

條件由上而下比對，第一個符合的分支執行後即結束判定。

計數與條件迴圈：for / while

```
for n in range(1, 10):
    if n % 2 == 0: print(n)

continue 跳過本次迭代；break 跳出迴圈。
```

list、tuple、dict、set

型別	特性	常用語法範例	適用場景
list 串列	有序、可變序列	<pre>fruits = ["apple", "banana"] fruits.append("orange") fruits.pop()</pre>	循序存取，並用 append、pop 增減
tuple 元組	有序、不可變	<pre>point = (3, 4)</pre>	儲存固定資料
dict 字典	鍵值對	<pre>user = {"name": "Amy", "age": 20} user["name"]</pre>	依鍵讀取對應資料
set 集合	無重複集合	<pre>set([1, 2, 2, 3]) a & b</pre>	去重與交集運算

Pythonic 技巧 • 串列推導式 (List Comprehension)：[x*x for x in range(10)]，用一行建立新串列。

函式設計與模組拆分

- 用 def 定義函式：`def add(a, b=0): return a + b`
- 支援預設參數、`*args`、`**kwargs` 與型別提示
- lambda 適合簡短的匿名函式，如 `sorted(data, key=lambda x: x[1])`
- 使用標準函式庫：`import math`、`from datetime import datetime`
- 自己的 .py 檔即是模組，可被其他檔案 import

標準函式庫精華

善用 `math`、`datetime` 與 `random`，無須安裝第三方套件即能滿足日常需求。

檔案讀寫與例外處理

現代 Python 堅持「資源自動釋放」與「明確捕捉錯誤」，確保程式遇到意外時優雅處置。

檔案安全讀寫：with 語句

```
import json

data = {"status": "ok"}

with open("config.json", "w", encoding="utf-8") as f:
    json.dump(data, f, ensure_ascii=False)

with open("config.json", encoding="utf-8") as f:
    loaded = json.load(f)
```

指定 `encoding="utf-8"` 能防止中文亂碼。

防禦式程式設計：try / except

```
def parse_age(text: str) -> int:
    try:
        age = int(text)
        if age < 0:
            raise ValueError("年齡不能為負數")
        return age
    except ValueError as err:
        print(f"輸入錯誤: {err}")
        return 0
```

精準捕捉特定例外，避免使用寬鬆的 `except:`。

第三方套件與實作練習

requests

抓取網頁 API：`requests.get(url).json()`

pandas 與 matplotlib

pandas 處理表格資料，matplotlib 畫圖。

BeautifulSoup

解析 HTML，做簡單爬蟲。

小專案建議

記帳程式，結合函式、檔案讀寫與例外處理。

類別、繼承與封裝設計

以物件為中心建構大型專案，透過繼承與多型減少冗餘程式碼。

類別、繼承與多型

class 封裝屬性與方法；`__init__` 為建構函式，`self` 指向實例。不同類別實作同名方法，呼叫端無需判斷型別。

`Animal.__init__(name: str)` 設定 `self.name`；

`Animal.speak()` -> str 可拋出

`NotImplementedError`。 `Dog(Animal)` 接收 `name: str`、`breed: str`，以 `super().__init__(name)` 呼叫父類別並設定 `self.breed`；`speak()` 回傳 `f"{self.name} 汪汪叫"`。

修飾器與資料類別

`@property`、`@classmethod`、`@staticmethod` 與 `dataclass` 簡化樣板程式碼。

`from dataclasses import dataclass`；以 `@dataclass`

定義 `Account`，包含 `owner: str`、`_balance: float =`

`0.0`。 `@property balance` -> float 回傳

`self._balance`；`@classmethod create_vip(cls, owner: str)` 回傳 `cls(owner, 10000.0)`。

`@dataclass` 自動產生 `__init__` 與 `__repr__`，省去手寫樣板代碼。
Python 3.7+ 起可用 `dataclass` 模組。

迭代器、生成器與裝飾器

生成器用 `yield` 逐步產出資料，節省記憶體；裝飾器用 `@` 語法套用，分離計時等關注點。

生成器 (Generator)

逐步讀取，適合串流或 GB 級檔案；範例以 `read_large_log(file_path)` 開啟 `"server.log"`，用 `open(file_path, "r", encoding="utf-8")` 逐行檢查 `"ERROR"`，再以 `yield line.strip()` 產出，透過 `for err` 與 `print(f"警報: {err}")` 隨取隨用。

裝飾器 (Decorator) 與 context manager

`@timer` 計算函式執行時間：`import time`、`from functools import wraps`；`timer(func)` 以 `@wraps(func)` 包裝 `wrapper(*args, **kwargs)`，記錄 `t0 = time.perf_counter()`，呼叫 `func(*args, **kwargs)` 得到 `result`，再算 `duration = time.perf_counter() - t0`，印出 `f"{func.__name__} 耗時: {duration:.4f}s"` 並回傳。套用於 `heavy_task()`，其內容為 `sum(range(1000000))`。context manager (`with`) 可用 `__enter__` / `__exit__` 或 `contextlib` 自訂。

非同步、多執行緒與效能調校

並行方案	適用場景	實踐語法核心	GIL 限制考量
asyncio	大量高並發網路請求、I/O 等待	<code>async def fetch()</code> <code>await asyncio.gather(...)</code>	單執行緒事件循環，無鎖負擔
threading	檔案寫入、外部系統輪詢監控	<code>from concurrent.futures import</code> <code>ThreadPoolExecutor</code>	受限於 GIL，適合 I/O 密集
multiprocessing	影像轉換、數值矩陣等 CPU 運算	<code>from multiprocessing import Pool</code> <code>with Pool() as p: p.map(...)</code>	獨立進程完全繞過 GIL

效能調優原則：永遠先以 `cProfile` 或 `timeit` 測量真實瓶頸再針對性優化；優先選用標準函式庫 `collections.deque` 或 `itertools` 內建 C 語言實作。

測試、打包與專案規格

品質保證：pytest 與靜態檢查

自動化測試確保每次修改不引發回歸錯誤。

```
# test_app.py
def add(a: int, b: int) -> int:
    return a + b

def test_add():
    assert add(1, 2) == 3
```

用 `ruff` 做超高速檢查，搭配 `mypy` 檢查型別。

專案現代規範：pyproject.toml

PEP 621 以單一檔案整合元資料與依賴清單。

```
[project]
name = "my-awesome-tool"
version = "0.1.0"
dependencies = ["requests>=2.31"]

[build-system]
requires = ["hatchling"]
```

用 `build` 與 `twine` 發佈套件至 PyPI。

學習路線與下一步

每週實作一個小專案，比只看教學更能內化

資料分析

使用 pandas。

網站與自動化

網站（Flask、Django）與自動化。

機器學習

使用 scikit-learn、PyTorch。

推薦資源：官方文件 docs.python.org • Python 官方 Tutorial •
LeetCode 練習

